

Teoría de autómatas y lenguajes formales

Autómatas y compiladores

Segundo semestre 2024

IIC2223 / IIC2224

Prof. Cristian Riveros

¿teoría de autómatas?

Automata theory

From Wikipedia, the free encyclopedia

Automata theory is the study of **abstract machines** and automata, as well as the **computational problems** that can be solved using them.

abstract machines: *“máquinas que NO son implementadas pero que son definidas matemáticamente”*

computational problems: compiladores, extracción de información, bases de datos, verificación de software, ... ;

¿algunos ejemplos de **máquinas abstractas**?

... daremos dos ejemplos.

```
    a = replaceAll(" ", " ", a); a = a.replace(
    return a.split(" "); } $("#unique")
function() { var a = array_from_string($("#fin").
    .val()), c = use_unique(array_from
    .val())); if (c < 2 * b - 1) { re
    * c), this.trigger("click"); } fe
    { "" != a[b] && "" != a[b] || a
    .val()); c = array_from
    c.length; b++) { -1 != a.indexOf
    for (b = 0; b < c.length; b++)
    } } $("#User_logged").val(a)
    this.click(function()
```

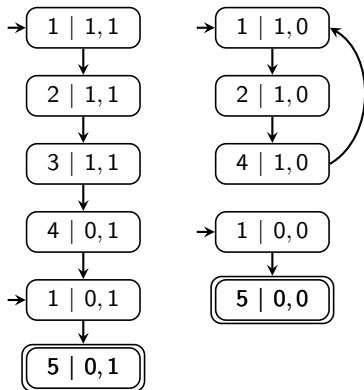
Máquina abstracta que representa un programa

```
1 while  $x = 1$  do
2   if  $y = 1$  then
3      $x \leftarrow 0$ 
4      $y \leftarrow (1 - x)$ 
5 return
```

$l \mid x, y$

$l \in \{1, \dots, 5\}$

$x, y \in \{0, 1\}$



¿siempre se detiene este código?

Máquina abstracta que representa programas concurrentes

```
1 while true do
2   if  $x < 2$  then
3      $x \leftarrow x + 1$ 
```

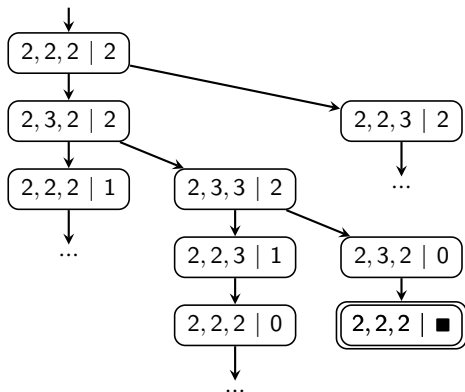
```
1 while true do
2   if  $x > 0$  then
3      $x \leftarrow x - 1$ 
```

```
1 while true do
2   if  $x = 2$  then
3      $x \leftarrow 0$ 
```

$l_1, l_2, l_3 \mid x$

$l_1, l_2, l_3 \in \{2, 3\}$

$x \in \{0, 1, 2, \blacksquare\}$



¿siempre se cumple que $0 \leq x \leq 2$?

Model checking

Model checking: dado un modelo de un sistema, verificar automáticamente si el modelo cumple una especificación dada.

Dos premios Turing (novel en computación) en esta área:

- Amir Pnueli (1996).

"For seminal work introducing temporal logic into CS and for outstanding contributions to program and systems verification."

- Edmund M. Clarke, E. Allen Emerson y Joseph Sifakis (2007).

"For their roles in developing model checking into a highly effective verification technology, widely adopted in the hardware and software industries."

Teoría de autómatas constituye una parte fundamental de model checking y verificación formal de software.



Secuencias de ADN y verificación de patrones

AGGATGACCCGAAATGCCCC
TCCAGCCAAAGGAGTCCGTT
ATGAGGGGATGGCAGCATGT
TGGTGGACAATTTTCGAGGGA
GAGAACCGCTTAGCAGCGCT
TTTGACCGAAATAACCCATA
GCCTCGCAATAATAGTACGC
CGCAATGAAGCTTGTTTGAG
TCTTAACAGTATCTGGA...

Buscamos el patrón “**ACAA**”.

Secuencias de ADN y verificación de patrones

AGGATGACCCGAAATGCCCC
TCCAGCCAAAGGAGTCCGTT
ATGAGGGGATGGCAGCATGT
TGGTGG**ACAA**TTTCGAGGA
GAGAACCGCTTAGCAGCGCT
TTTGACCGAAATAACCCATA
GCCTCGCAATAATAGTACGC
CGCAATGAAGCTTGTTTGAG
TCTTAACAGTATCTGGA...

Buscamos el patrón "**ACAA**".

```
1 for  $i \leftarrow 1$  to  $d.length$  do
2   if  $d[i] = 'A'$  then
3     if  $d[i + 1] = 'C'$  then
4       if  $d[i + 2] = 'A'$  then
5         if  $d[i + 3] = 'A'$  then
6           Output  $i$ 
```

Para un documento d y
un patrón p

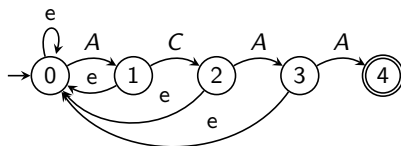
tiempo: $O(|d| \cdot |p|)$

¿podemos hacer una algoritmo más eficiente?

Secuencias de ADN y verificación de patrones

AGGATGACCCGAAATGCCCC
TCCAGCCAAAGGAGTCCGTT
ATGAGGGGATGGCAGCATGT
TGGTGG**ACAA**TTTCGAGGA
GAGAACCGCTTAGCAGCGCT
TTTGACCGAAATAACCCATA
GCCTCGCAATAATAGTACGC
CGCAATGAAGCTTGTTTGAG
TCTTAACAGTATCTGGA...

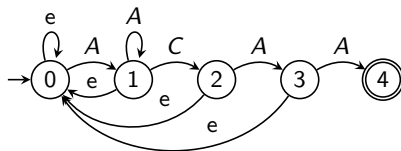
```
1 s = 0
2 for i ← 1 to d.lenght do
3   if s = 0 ∧ d[i] = 'A' then
4     s = 1
5   else if s = 1 ∧ d[i] = 'C' then
6     s = 2
7   else if s = 2 ∧ d[i] = 'A' then
8     s = 3
9   else if s = 3 ∧ d[i] = 'A' then
10    Output i - 3
11  else s = 0
```



Secuencias de ADN y verificación de patrones

AGGATGACCCGAAATGCCCC
TCCAGCCAAAGGAGTCCGTT
ATGAGGGGATGGCAGCATGT
TGGTGGACAAATTTTCGAGGA
GAGAACCGCTTAGCAGCGCT
TTTGACCGAAATAACCCATA
GCCTCGCAATAATAGTACGC
CGCAATGAAGCTTGTTTGAG
TCTTAACAGTATCTGGA...

```
1 s = 0
2 for i ← 1 to d.lenght do
3   if (s = 0 ∨ s = 1) ∧ d[i] = 'A' then
4     s = 1
5   else if s = 1 ∧ d[i] = 'C' then
6     s = 2
7   else if s = 2 ∧ d[i] = 'A' then
8     s = 3
9   else if s = 3 ∧ d[i] = 'A' then
10    Output i - 3
11  else s = 0
```



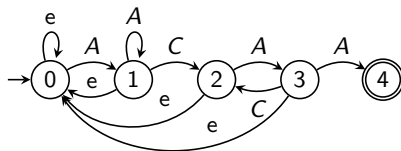
Secuencias de ADN y verificación de patrones

AGGATGACCCGAAATGCCCC
TCCAGCCAAAGGAGTCCGTT
ATGAGGGGATGGCAGCATGT
TGGTGG**ACAA**TTTCGAGGA
GAGAACCGCTTAGCAGCGCT
TTTGACCGAAATAACCCATA
GCCTCGCAATAATAGTACGC
CGCAATGAAGCTTGTTTGAG
TCTTAACAGTATCTGGA...

Para un documento d y
un patrón p

tiempo: $O(|d| + |p|)$

```
1  $s = 0$ 
2 for  $i \leftarrow 1$  to  $d.length$  do
3   if  $(s = 0 \vee s = 1) \wedge d[i] = 'A'$  then
4      $s = 1$ 
5   else if  $(s = 1 \vee s = 3) \wedge d[i] = 'C'$  then
6      $s = 2$ 
7   else if  $s = 2 \wedge d[i] = 'A'$  then
8      $s = 3$ 
9   else if  $s = 3 \wedge d[i] = 'A'$  then
10    Output  $i - 3$ 
11  else  $s = 0$ 
```



Pattern matching y teoría de autómatas

Pattern matching: encontrar la presencia de un patrón en una secuencia de información.

Varias aplicaciones en:

- Secuenciación de ADN.
- Búsquedas en la web.
- Lingüística.
- ...

Teoría de autómatas constituye el modelo de abstracción para el desarrollo de algoritmos más eficientes.



FINISHED (FOUND 84)

MULTI

1 >XP_017210497.1 glutamate [FMDA] receptor subunit
epsilon-2 isoform X1 [Danio rerio]
2 MGVLAMFKGLYLHSSAMLSLHSSSPFSDCRVLSFVSLVSSFKLPLYSLILLLS
LFFFTTCELVTMNEPKSIINRITDPKSIINI CALMSRNWLQGVFGESRR **GGGIGT**
PTGGMNSQSIISPPHYPPPGVSPVGPMPSPKFAQGLSIAVILVGNSEVSLSEGLE
KEDFLHVLPLPPKILVTMTGMDPAKSIINRICALMSRNWLQGVFGDDTDQEAIAQ
LDFISAQTHPIPVILGRGSSMIAKDDHSMFFQGPSIEQQASVMLNIMEEYDW
IFSIVTTYPGHQDFVNRIRSTVDNSFVGWLEEVLLDMSVDGDSKIKQGMKKL
QSPVILLYCTKEEATTIFEVAHSVGLTGYGYTWIVPSLVAGD TDNPVNVFPTGLIS
VSYDWDYVGLEARVDAVAIIAMATSTMMLDRGPHTL LKSGCHGAPDKKGSKGNP
NEVLYRLMNVTFEGRNLSFSEDGFMQHPLVILLNKRQYERQVKGWENGLAMKY
HVMRFPEYLSDAEEREDDHLVTLEAPVIEVDLPSTGCMRNTVPCRKQLKL
QNLTDGSGIYIKRCKGKFCIDILKKIAKSVKFTYDYLVLVTNGKHGKKINGTWNMGV
GEVVLKNAHMAVGSLTINEERSEVDISVPVF IETGISVMVSRNGTVSPSALEPFS
SADVVMVMFVMLLIVSAVAVFVEYFSPVGNRCLADGREPGSPFTI KAIWLW
GLVFNNSPVNPKGKTSKIMVSVWAFFAVILFASYTANLAAFMIEQEEYVDQVTGL
SDKKLOHPNDPSFPFFRFTVPVNGSTERNIRNNYKEMHSYMTSFHQKNVNEALHSLK
SGKLDAFYTDAAVLNVMAGRDGCKLVTIGSGYIFATTGYGIAIGVSGDKWKRAVDL
AIQLFDGDEMEALEALWLGTICHNEKNEMVSSQLVDVNMGAGVFMYLGAAMLSIT
TFIAEHLFYWLRLFCFMGVCSKPGKMTFSISRGIYSICIHGVQIEENKSALNSPST
MKMMNMNTHSNILRLRTAKNMTSPVGVNGSPHSALDYSHRESAVVDISEHRRSLA
GHSDCKPPPLYPEDNMFSDYVESEVTERFGNHLKDSNLQDHYHLHHGSEALGAM
SGPLPNRPSLGSASSLEGGVDCSLGGGVAPITFTTPQRS LTHNRKEFDLIAGH
PTQSSGSKGLPDLYGKFSGKGGASSGFIAGHDRYCGGGVGSGGDDGNIRSDVSD
ISTHTVTYGNLGHGSKRRRKYQRDLSLKKRPASAKSRREQDETELGRFRPHHTIIHH
LHHPLPATAQRSHTPVERKSRQGNCTSYLFRDKNLENDRFYVDQFRAKESGAWD
LDLSDAPGGGGVGLGGSGCGVVSSGAGACTSLVPMEDFLKGGSKCTCKGGM
GGSGPGQQGHACWEKIGGVGGLAGGDWECRSCSHSGGVSGGSGKPCVMHGGGGAGG
YPAAGGVGGSGSGSGSISRPSSATCTKRCDSCKKPNLYDISEDNLLLDQIAGKHP
LESKGGGGGTGAQTQVQRKKFGPGGKVLRRQHSYDITFVELQKAGAGRMGGFGGGGG
ASMLPPPRSVLSDKDDRYMEGASPYAQMFQEQYAGGERETSYFGDRGKGGGSSFLS
RGGEGGLHRRSVGERDMRDRDRGMMGGGVGGTRGVGTYLKSKSLYDPKVNQNPFT

Match 0

Match 0

motif (86-92) GVGLAM

Match 1

motif (190-196) GGGIGT

Match 2

motif (194-200) **GTPTGG**

Match 3

motif (198-204) GGMNSO

Match 4

motif (230-236) GLSIAV

Match 5

motif (292-298) GVVFGD

Match 6

motif (321-327) GIRGGS

Match 7

motif (525-531) GSKSGN

Match 8

motif (693-699) GTWNGM

Match 9

motif (812-818) GLVFNN

CONCLUSIONES

Algunas conclusiones de los dos ejemplos

1. Autómatas constituyen una noción fundamental y útil en ciencia de la computación.
2. Aplicaciones de teoría de autómatas en diversas áreas de computación.
3. La modelación con teoría de autómatas nos permite abstraernos del problema para así encontrar soluciones más eficientes.

*"I suggest that automata theory is the **linear algebra** of computer science. . . as a basic, fundamental subject, known and used by everyone, which has formed part of the intellectual landscape for so long that it is no longer noticed."*

Jacques Sakarovitch.

Outline

Motivación

Programa

Programa del curso



Cristian Riveros

- Profesor
- cristian.riveros@uc.cl
- Oficina 3-S, DCC



Cristóbal Rojas

- Ayudante jefe
- cristobalrojas@uc.cl



Amaranta Salas

- Ayudante de cátedra
- afsalas@uc.cl

Programa del curso

Clases: Lunes y Miércoles módulo 2.
Sala AM2 (por confirmar).

Ayudantías: Viernes módulo 2.
Sala A5.

Atención: A toda hora, preferentemente después de clases.

Tanto las clases como las ayudantías serán presenciales y sin asistencia

Objetivos del curso

1. Estudiar las nociones fundamentales de la ciencia de la computación.
2. Entender los **modelos básicos** de computabilidad y complejidad.
3. Entender la importancia para aplicaciones prácticas en ingeniería.

Temario del curso



Evaluaciones

1. Tareas.
2. Interrogaciones.
3. Examen.

Tareas

- Seis tareas durante el semestre (se borrará la peor tarea).
- Enunciado se publicará los días viernes.
- La entrega será el jueves siguiente hasta las 23:59.
- La evaluación de cada pregunta en las tareas será de:
 - 0 (respuesta incorrecta)
 - 1 (con errores importantes)
 - 2 (con errores)
 - 3 (con errores menores)
 - 4 (respuesta correcta)

Tareas: formato de entrega

- El método de entrega será online.
- La tarea debe ser escrita y entregada en $\text{\LaTeX}$.

NO se aceptarán tareas **escritas a mano**
ni en otro sistema de composición de texto

Tareas: problema excepcional

- Cada estudiante cuenta con un cupón #problemaexcepcional.
- Se puede utilizar **solo una vez durante el semestre**.

Permite extender el plazo de entrega de una tarea sin necesidad de una justificación debido a motivos excepcionales y personales.

- La extensión mueve el plazo de entrega hasta las 23:59 horas del día lunes de la semana siguiente a la entrega original.

NO se aceptarán tareas **fuera de plazo** y **NO** se harán **excepciones**.

Interrogaciones y examen

- Dos interrogaciones y un examen.
- Interrogaciones presenciales a las 17:30 y duración de 2 a 3 horas.
- Examen presencial a las 13.30 horas.

Interrogaciones y examen: fechas

	Publicación enunciado	Entrega
Tarea 1	Viernes 16 de Agosto	Jueves 22 de Agosto
Tarea 2	Viernes 6 de Septiembre	Jueves 12 de Septiembre
Interrogación 1	Viernes 27 de Septiembre	
Tarea 3	Viernes 4 de Octubre	Jueves 10 de Octubre
Tarea 4	Viernes 18 de Octubre	Jueves 24 de Octubre
Interrogación 2	Lunes 28 de Octubre	
Tarea 5	Viernes 8 de Noviembre	Jueves 14 de Noviembre
Tarea 6	Viernes 22 de Noviembre	Jueves 28 de Noviembre
Examen	Miércoles 4 de Diciembre	

“El profesor no se hará responsable por tope de horarios con interrogaciones o exámenes de otros cursos.”

Interrogaciones y examen: problema de fuerza mayor

Problema de **fuerza mayor** = cualquier enfermedad o problema que impida rendir una evaluación.

Para solo una **interrogación**:

- La nota del examen reemplazará la nota de esta interrogación.
- La inasistencia **NO** necesita ser justificada.
 - Se reemplazará **automáticamente** la peor nota por el examen.

Para el **examen**:

- Justificativo según las reglas de la Escuela de Ingeniería en Pregrado.
- Nota P y el examen se rinde a comienzos del próximo semestre.

NO se harán excepciones.

Evaluación

Si $N_1, \dots, N_k$ es una lista de k notas:

$AVG_n(N_1, \dots, N_k) \quad := \quad$ promedio aritmético de los n valores
más altos de $N_1, \dots, N_k$.

Nota **Tareas** (PT):

$$PT = AVG_5(T_1, T_2, T_3, T_4, T_5).$$

Nota **Interrogaciones** y **Examen** (PE):

$$PE = AVG_3(I_1, I_2, E, E)$$

Evaluación

Nota **Final** (**NF**):

$$\mathbf{NF} = 0,3 \cdot \mathbf{PT} + 0,7 \cdot \mathbf{PE}$$

El curso se aprueba si, y solo si, **todas** las siguiente condiciones se cumplen:

- **PT** $\geq 2,95$
- **PE** $\geq 3,95$
- **NF** $\geq 3,95$

En caso de **no aprobar**, la nota final del curso será $\min\{\mathbf{NF}, 3,9\}$.

Examen recuperativo de última instancia

*"Como última instancia para aprobar el curso en caso de reprobación por las reglas anteriores, se dará la posibilidad de un **examen recuperativo oral**."*

1. Reprobación por Interrogaciones y Examen:

$$3.95 > \mathbf{PE} \geq 3.65 \quad \text{y} \quad \mathbf{PT} \geq 3.95$$

2. Reprobación por Tareas:

$$\mathbf{PE} \geq 3.95 \quad \text{y} \quad 2.95 > \mathbf{PT} \geq 1.95$$

3. Reprobación por promedio final:

$$\mathbf{PE} \geq 3.95 \quad \text{y} \quad \mathbf{PT} \geq 2.95 \quad \text{y} \quad 3.95 > \mathbf{NF}$$

Si el estudiante aprueba esta instancia, su nota final en el curso será 4.0.

Sobre correcciones y recorrecciones

El proceso de **corrección** y **recorrección** de evaluaciones será el siguiente:

1. Entrega de notas y feedback (online): 2 semanas.
2. Recorrección “presencial”.
3. Recorrección “escrita”.
4. En caso de no quedar satisfecho con recorrección, solicitar la recorrección con el profesor (enviar un correo).

En caso de tener cualquier duda sobre corrección o feedback,
enviar un correo a **iic2223@ing.puc.cl**.

Sobre copia

- Tanto las tareas, interrogaciones y examen son individuales.
- Material de libros o Internet debe estar debidamente **referenciados**.
- En **caso de copia**, los antecedentes serán enviados a la DiPre para evaluar posteriores sanciones en conjunto con la Universidad, las que pueden incluir reprobación del curso y un procedimiento sumario.

Comunicación digital

- Anuncios, clases, ejercicios, etc . . .

Canvas / Teoría de Autómatas y Lenguajes Formales

- Preguntas sobre materia:

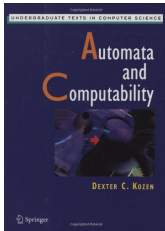
- Foro de Canvas.
- Personales:

iic2223@ing.puc.cl

- Preguntas por problemas personales relacionados al curso:

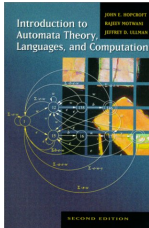
cristian.riveros@uc.cl

Bibliografía



Automata and Computability

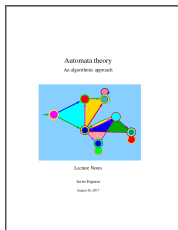
- Dexter Kozen
- 1997



Intr. to automata theory languages and computation

- Hopcroft, Motwani, y Ullman
- Múltiples ediciones: 1979, 2000, 2006

Bibliografía



Automata Theory: An Algorithmic Approach.

- Javier Esparza
- 2023



Compiler design: syntactic and semantic analysis

- Wilhelm, Seidl y Hack
- 2013

¿PREGUNTAS?